Large Language Models

Complete Learning Notes: From Transformer to LangChain

Week 1: Fundamentals → Week 2: Fine-tuning → Week 3: Alignment → Week 4: Applications

Aiden Tan

January 16, 2026

Contents

1 Course Overview

Week 1	Week 2	Week 3	Week 4
Transformer	Instruction Fine-tuning	RLHF	Model Optimization
Model Types	Full SFT vs LoRA	Reward Model	RAG
Pre-training	Evaluation & Benchmarks	PPO + KL Penalty	CoT / PAL
Quantization + ICL	PEFT Techniques	Preference Alignment	ReAct / LangChain

Who is this for?

If you're new to LLMs, this note will help you understand each concept with **plenty of examples**. We'll start from the fundamentals of Transformer and guide you all the way to RLHF and modern application frameworks.

Week 1: Transformer Architecture & Pre-training Fundamentals

Goal: Understand the backbone of LLMs — the Transformer architecture, and design principles of different model types.

2 Transformer Architecture

What is Transformer?

Transformer is a neural network architecture proposed by Google in 2017, serving as the foundation for all modern LLMs. Its core innovation is the **Self-Attention mechanism**, which allows the model to "see" all positions in the input sequence simultaneously, rather than processing sequentially like RNNs.

2.1 Core Components

Component	Function
Self-Attention	Computes relevance between each token and all other tokens
Multi-Head Attention	Multiple attention heads in parallel, capturing different relationships
Feed-Forward Network	Applies non-linear transformation to each position independently
Layer Normalization	Stabilizes training and accelerates convergence
Positional Encoding	Injects position information (attention is position-agnostic)

Example:

Intuitive Understanding of Self-Attention Input sentence: `"The cat sat on the mat"`
When processing `"sat"`, Self-Attention computes its relevance with every word:

- `"cat"` → High relevance (who sat? the cat)
- `"mat"` → Medium relevance (where? on the mat)
- `"The"` → Low relevance (article, less important)

This enables the model to understand semantic relationships between words.

2.2 Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q (Query): "What am I looking for?"
- K (Key): "What do I have that can be matched?"
- V (Value): "What content to return when matched?"
- $\sqrt{d_k}$: Scaling factor to prevent large dot products

3 Three Model Architecture Types

Transformer has three variants for different tasks:

Type	Structure	Best For	Examples
Encoder-Only	Encoder only	Understanding: classification, NER, sentiment	BERT, RoBERTa
Decoder-Only	Decoder only	Generation: text generation, dialogue	GPT series, LLaMA
Encoder-Decoder	Both	Seq2Seq: translation, summarization	T5, BART, FLAN-T5

3.1 Encoder-Only Models

Key Characteristics

- Uses **Bidirectional Attention**: each token sees all tokens (left and right)
- Excels at "understanding" tasks, not generation
- Pre-training: Masked Language Modeling (MLM)

Example:

BERT's MLM Pre-training Input: `"The [MASK] sat on the mat"`
Task: Predict what `[MASK]` is
Answer: `"cat"`
This teaches the model to understand contextual semantics.

3.2 Decoder-Only Models

Key Characteristics

- Uses **Causal/Masked Attention**: each token only sees tokens to its left
- Excels at "generation" tasks: predicts next token sequentially
- Pre-training: Next Token Prediction

Example:

GPT's Autoregressive Generation Prompt: `"The weather today is"`
Generation process (token by token):

1. Input `"The weather today is"` → Predict `"sunny"`
2. Input `"The weather today is sunny"` → Predict `"and"`
3. Input `"The weather today is sunny and"` → Predict `"warm"`
4. ...continues until end token

3.3 Encoder-Decoder Models

Key Characteristics

- Encoder "understands" input, Decoder "generates" output
- Encoder uses bidirectional attention, Decoder uses causal attention
- Connected via Cross-Attention
- Best for tasks where input/output formats differ

Example:

FLAN-T5 for Summarization **Input (to Encoder):**
`"Summarize: The company reported strong Q3 earnings, beating analyst expectations by 15%. Revenue grew 20% year-over-year, driven by cloud services. The CEO announced plans for expansion into Asian markets."`
Output (Decoder generates):
`"Company exceeds Q3 expectations with 20% revenue growth from cloud services, plans Asian expansion."`

What is FLAN-T5?

FLAN-T5 = T5 model + Instruction Fine-tuning
FLAN stands for "Fine-tuned Language Net" — Google fine-tuned T5 on massive instruction data, making it better at following instructions. It's the main model used in our labs.

4 Pre-training

What is Pre-training?

Pre-training uses **large-scale unlabeled text** to train the model. The goal is to learn fundamental language patterns: grammar, semantics, world knowledge. After pre-training, the model has general language abilities but doesn't know how to "follow instructions."

4.1 Pre-training Objectives

Model Type	Objective	Method
Encoder-Only	MLM	Randomly mask 15% of tokens, predict them
Decoder-Only	Next Token	Given prefix, predict the next token
Encoder-Decoder	Span Corruption	Mask contiguous spans, recover them

Example:

T5's Span Corruption **Original:** `"The quick brown fox jumps over the lazy dog"`
Corrupted Input: `"The <X> fox <Y> the lazy dog"`
Target Output: `"<X> quick brown <Y> jumps over"`
The model learns to recover masked spans.

5 Model Quantization

Why Quantization?

Large models have billions of parameters (GPT-3 has 175B). Using FP32 directly:

- Huge memory footprint (175B × 4 bytes = 700GB)
- Slow inference
- High hardware costs

Quantization reduces numerical precision to compress models and speed up computation.

5.1 Precision Comparison

Precision	Size per Param	Range	Use Case
FP32 (Single)	4 bytes (32 bits)	High precision	Training (default)
FP16 (Half)	2 bytes (16 bits)	Medium	Training/Inference
BF16	2 bytes (16 bits)	Wider range, less precision	Training (stable)
INT8	1 byte (8 bits)	Integer, limited	Inference deployment
INT4	0.5 bytes (4 bits)	Very limited	Extreme compression

Example:

Quantization in Practice LLaMA-7B example:

Precision	Model Size	Memory Required
FP32	28 GB	~32 GB
FP16	14 GB	~16 GB
INT8	7 GB	~8 GB
INT4	3.5 GB	~4 GB

From FP32 to INT4, model size reduces by 8x!

6 In-Context Learning (ICL)

Definition

In-Context Learning allows the model to perform tasks **without updating parameters** — just by providing task descriptions and examples in the prompt.
This is one of the most remarkable abilities of LLMs — you don't need training, just "tell" the model what to do.

6.1 Three Modes

Mode	Definition	Prompt Example
Zero-shot	Instruction only, no examples	`"Translate to French: Hello"`
One-shot	1 example provided	`"English: Hello → French: Bonjour` `English: Goodbye → French: ?"`
Few-shot	3-10 examples provided	Multiple input-output pairs

Example:

Few-shot Sentiment Classification **Prompt:**

```
Classify the sentiment as positive or negative.

Review: "This movie was amazing!" → Sentiment: positive
Review: "Terrible waste of time." → Sentiment: negative
Review: "Best purchase I ever made!" → Sentiment: positive

Review: "I regret buying this product." → Sentiment:
```

Model Output: `negative`
The model "learns" classification rules from examples — no training needed!

6.2 ICL Pros and Cons

Pros	Cons
No training required, zero cost	Sensitive to prompt format
Fast iteration (just modify prompt)	Limited by context length
Great for rapid prototyping	Example order can affect results
No data pipeline needed	Usually worse than fine-tuning

Week 1 Key Takeaways

1. **Transformer** is the foundation of all LLMs, with Self-Attention as its core
2. **Three architectures**: Encoder-Only (understanding), Decoder-Only (generation), Encoder-Decoder (seq2seq)
3. **Pre-training** teaches general language abilities, but not instruction-following
4. **Quantization** (FP16/INT8) significantly reduces computational costs
5. **ICL** enables task completion via prompt examples, no training required

Week 2: Instruction Fine-tuning & Parameter-Efficient Methods

Goal: Learn how to make models "follow instructions" — from temporary ICL adaptation to actually changing model behavior through fine-tuning.

7 Why Fine-tuning?

Limitations of ICL

While ICL is convenient, it has fundamental limitations:

- Parameters unchanged — behavior change is "temporary"
- Must repeat examples every inference, wasting tokens
- Complex tasks perform worse than fine-tuning
- Cannot make the model "remember" certain behavior patterns

Fine-tuning updates model parameters, making behavior changes "permanent."

8 Instruction Fine-tuning (SFT)

What is Instruction Fine-tuning?

Instruction Fine-tuning (also called Supervised Fine-tuning, SFT) trains the model with "instruction-response" pairs, teaching it to:

- Understand various instruction formats
- Generate appropriate responses
- Refuse inappropriate requests

This is the key step transforming a pre-trained model into an "assistant."

8.1 Training Objective

$$\max_{\theta} \sum_{(x,y)\in D} \log \pi_\theta(y \mid x)$$

- x: instruction + input
- y: expected output (ground truth)
- π_θ: model's output probability distribution
- Goal: maximize probability of generating correct answers

9 Full Fine-tuning vs PEFT (LoRA)

9.1 Full Fine-tuning

Full Fine-tuning

Updates **all** (or most) model parameters.

Pros	Cons
Best performance, strongest expressiveness	High compute cost (massive GPU needed)
High data distribution fit	Prone to overfitting
Can learn complex task patterns	Catastrophic Forgetting
	Need to store full model per task

Catastrophic Forgetting

When training on a new task, the model may "forget" general abilities learned during pre-training.
Example: After fine-tuning for summarization, the model might become worse at translation.

9.2 PEFT: Parameter-Efficient Fine-Tuning

PEFT Core Idea

Don't update all parameters — train only a **small number of extra parameters** to achieve near full fine-tuning performance.
Benefits:

- Dramatically reduced memory requirements
- Faster training
- Can save different small adapters for different tasks
- Reduced catastrophic forgetting

9.3 LoRA: Low-Rank Adaptation

LoRA Principle

LoRA's key insight: weight changes during task adaptation are **low-rank**.
Therefore, we can approximate weight changes with the product of two small matrices:

$$W' = W + \Delta W = W + BA$$

- $W \in \mathbb{R}^{d\times d}$: Original weights (frozen, not updated)
- $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$: LoRA matrices (trainable)
- $r \ll d$: rank, typically 4, 8, 16, or 32
- Parameters reduce from d^2 to $2dr$ (e.g., $d = 4096, r = 8$ means 256x reduction)

Example:

LoRA Parameter Calculation Original attention weight $W_Q \in \mathbb{R}^{4096\times4096}$:
Full Fine-tuning:

- Trainable params = $4096 \times 4096 = 16,777,216$ (∼16.8M)

LoRA (rank=8):

- $A \in \mathbb{R}^{8\times4096}$: 32,768 params
- $B \in \mathbb{R}^{4096\times8}$: 32,768 params
- Total = 65,536 (∼65K, **256x reduction!**)

9.4 Full vs LoRA Comparison

Aspect	Full Fine-tuning	LoRA
Trainable Parameters	100%	0.1% – 1%
GPU Memory	High	Low
Training Speed	Slow	Fast
Multi-task Deployment	Store multiple full models	Same backbone + multiple adapters
Performance	Best	Close (usually minor gap)
Catastrophic Forgetting	Severe	Mild

10 Evaluation & Benchmarks

Why Evaluation?

After fine-tuning, we need to know if the model actually improved. Metrics help us:

- Quantify model performance
- Compare different models/methods
- Identify potential issues

10.1 Overlap-based Metrics

These metrics score based on n-gram overlap between generated text and reference.

10.1.1 BLEU (Bilingual Evaluation Understudy)

Core Idea: How many n-grams in generated text appear in reference (Precision)

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

- p_n: n-gram precision
- BP: Brevity Penalty (penalizes short outputs)
- Commonly used for machine translation

10.1.2 ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Core Idea: How many n-grams in reference are covered by generated text (Recall)
Common Variants:

- **ROUGE-1**: Unigram recall
- **ROUGE-2**: Bigram recall
- **ROUGE-L**: Based on Longest Common Subsequence (LCS)

Commonly used for summarization.

Limitations of Overlap-based Metrics

- Unfriendly to paraphrases (same meaning, different words = low score)
- Don't measure safety, helpfulness, truthfulness
- May not align with human preferences

This is why we need RLHF in Week 3!

Week 2 Key Takeaways

1. **Instruction Fine-tuning** teaches models to follow instructions
2. **Full Fine-tuning** performs best but is expensive with forgetting risk
3. **LoRA** uses low-rank decomposition, training only 0.1%-1% parameters
4. **BLEU** measures precision, **ROUGE** measures recall
5. Traditional metrics have limitations, can't measure safety and preferences

Week 3: RLHF — Aligning Models with Human Preferences

Goal: Understand how to use reinforcement learning to make models not just "correct" but "good" — safe, helpful, and aligned with human values.

11 Why RLHF?

Limitations of SFT

Instruction Fine-tuning makes models "follow instructions," but has issues:

- Ground truth answers aren't necessarily the "best" answers
- Can't express "this response is better than that" preferences
- May generate harmful, unsafe content
- May "confidently hallucinate" (make things up)

RLHF introduces human preferences, teaching models to generate what humans actually want.

Example:

SFT vs RLHF Difference **Question:** `"How do I pick a lock?"`
SFT model might answer: `"First, you need a tension wrench and pick. Insert the tension wrench..."` (Technically correct, but could enable illegal activity)
RLHF model might answer: `"I can't provide instructions for picking locks as it could enable illegal entry. If you're locked out, I recommend calling a licensed locksmith."` (Safer, more socially appropriate)

12 RLHF Training Pipeline

RLHF has three stages:

Stage	Name	What It Does
1	Supervised Fine-tuning	Train with ground truth for basic instruction-following
2	Reward Model Training	Collect human preferences, train a "scorer"
3	RL Fine-tuning (PPO)	Use Reward Model scores as rewards to optimize policy

13 Reward Model

Role of Reward Model

Reward Model is a "judge" that scores model outputs:

- Input: prompt + response
- Output: scalar score $r(x, y) \in \mathbb{R}$

- Higher score = response better aligns with human preferences

13.1 Training Data: Preference Pairs

Example:

Preference Data Collection **Prompt:** `"Explain quantum computing to a child."`
Response A: `"Quantum computing uses qubits that can be 0 and 1 simultaneously through superposition, enabling parallel computation..."`
Response B: `"Imagine a magic coin that can be heads AND tails at the same time! Quantum computers use these special coins to solve puzzles really fast."`
Human Label: B > A (B is better for explaining to a child)
This "A vs B" preference data trains the Reward Model.

13.2 Reward Model Training Objective

$$\mathcal{L}_{RM} = -\mathbb{E}_{(x,y_w,y_l)} \left[\log \sigma \left(r(x,y_w) - r(x,y_l)\right)\right]$$

- (x, y_w, y_l): prompt, preferred response, less preferred response
- $r(x, y)$: Reward Model's score
- σ: sigmoid function
- Goal: make $r(x, y_w) > r(x, y_l)$

14 PPO: Proximal Policy Optimization

Why PPO?

PPO is a stable policy gradient algorithm with features:

- Clipped objective prevents overly large updates
- More stable than vanilla policy gradient
- Widely used in RLHF

14.1 RLHF Optimization Objective

$$\max_{\theta} \mathbb{E}_{x\sim D, y\sim\pi_\theta(\cdot|x)} \left[r(x,y) - \beta \cdot \mathrm{KL}\left(\pi_\theta(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\right)\right]$$

- $r(x, y)$: Reward Model's score
- $\mathrm{KL}(\cdot\|\cdot)$: KL divergence, measures distribution difference
- π_{ref}: Reference model (usually SFT model, frozen)
- β: KL penalty coefficient

14.2 Core Components

Component	Symbol	Role
Policy Model	π_θ	Model being optimized, generates responses
Reference Model	π_{ref}	Frozen, provides KL anchor
Reward Model	$r(x, y)$	Scores responses
Value Head	$V(s)$	Estimates state value, computes advantage

15 Importance of KL Penalty

Why KL Penalty?

Without KL constraint, the model might:

- **Reward Hacking**: Find ways to "cheat" the Reward Model for high scores, but output quality drops
- **Mode Collapse**: Only generate certain fixed patterns
- **Distribution Shift**: Drift away from pre-trained language abilities

KL Penalty forces the model to stay close to the reference model, maintaining language quality.

Example:

Reward Hacking Example **Scenario:** Reward Model gives high scores for "polite" responses
Without KL constraint:
Model learns to add `"Thank you so much for asking! I really appreciate your question!"` to every response.
This gets high reward but isn't actually helpful — it's annoying.
With KL constraint:
Model is constrained to normal language patterns, won't excessively "please" the Reward Model.

Week 3 Key Takeaways

1. **RLHF** optimizes using human preferences (not ground truth)
2. **Reward Model** is the "judge" that scores responses
3. **PPO** is a stable policy optimization algorithm using Advantage to reduce variance
4. **KL Penalty** prevents reward hacking and distribution shift
5. RLHF makes models not just "correct" but "good, safe, helpful"

Week 4: Model Optimization

Goal: Learn how to make models smaller and faster (optimization), provide new knowledge without retraining (RAG), solve complex problems (reasoning techniques), and connect to external tools (Orchestration).

16 Model Inference Optimization

Why Optimize Inference?

Trained large models are often:

- Too big: Hard to deploy on edge devices or phones
- Too slow: High inference latency, poor user experience
- Too expensive: GPU costs are significant

We need to make models smaller and faster **without significantly sacrificing performance**.

16.1 Three Optimization Techniques

Technique	Core Idea	Pros	Cons
Quantization	Reduce numerical precision	Simple, effective	Extreme quant. hurts
Pruning	Remove unimportant params	High compression	Needs retraining
Distillation	Small model learns from large	Good small model perf.	Requires training

16.2 Pruning

Pruning Principle

Many neural network parameters contribute little to the output (close to 0). Pruning **removes these unimportant parameters** to compress the model.
Analogy: Like pruning a tree — remove excess branches, the tree still grows healthy.

Pruning Type	Description
Unstructured	Remove individual weights (sparse matrix), high compression but hard to accelerate
Structured	Remove entire neurons/channels/layers, hardware-friendly but lower compression

16.3 Knowledge Distillation

Distillation Core Idea

Train a **small model (Student)** to mimic the behavior of a **large model (Teacher)**.
Key insight: Teacher's output probability distribution (soft labels) contains richer information than hard labels.

Example:

Why Soft Labels Work Better **Classification task:** Is this image a cat or dog?
Hard Label: `[1, 0]` (100% cat)
Teacher's Soft Label: `[0.9, 0.1]` (90% cat, 10% dog)
Soft Label tells Student: "This image mostly looks like a cat, but also somewhat like a dog" — this "dark knowledge" helps Student learn better.

Distillation Loss:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{CE}(y, \hat{y}_{student}) + (1 - \alpha) \cdot \mathcal{L}_{KL}(p_{teacher}, p_{student})$$

- $\mathcal{L}_{CE}$: Cross-entropy with ground truth
- $\mathcal{L}_{KL}$: KL divergence with Teacher's output
- α: Balance coefficient

Example:

Distillation Case Study **DistilBERT:**

- Teacher: BERT-base (110M parameters)
- Student: DistilBERT (66M parameters, 40% reduction)
- Performance: Retains 97% of BERT's performance
- Speed: 60% faster

17 RAG: Retrieval-Augmented Generation

Why RAG?

LLM knowledge has limitations:

- **Knowledge cutoff**: Training data has a cutoff date, doesn't know recent info
- **Private data**: Doesn't know your company's internal documents
- **Hallucination**: May fabricate non-existent information
- **Update cost**: Retraining is too expensive

RAG retrieves relevant documents at inference time, providing LLM with latest/private knowledge without retraining!

17.1 RAG Workflow

Step 1: User Query
`"What is our company's vacation policy?"`
↓
Step 2: Retrieve Relevant Documents
Find most relevant document chunks from knowledge base
↓
Step 3: Build Augmented Prompt
`Context: [Retrieved document content]`
`Question: What is our company's vacation policy?`
↓
Step 4: LLM Generates Answer
Generate accurate answer based on retrieved context

17.2 RAG Core Components

Component	Role
Document Loader	Load various document formats (PDF, Word, web, etc.)
Text Splitter	Split long documents into small chunks
Embedding Model	Convert text into vector representations
Vector Store	Store and retrieve vectors (FAISS, Pinecone, etc.)
Retriever	Find most relevant chunks for a query
LLM	Generate final answer based on retrieved content

18 Advanced Reasoning Techniques

LLM Reasoning Limitations

Standard LLMs struggle with complex reasoning:

- Multi-step math calculations often go wrong
- Long logical chains can "get lost"
- No "thinking process" — jumps straight to answer

These techniques improve reasoning by changing how we prompt or introducing external tools.

18.1 Chain of Thought (CoT)

CoT Core Idea

Make the model **show its reasoning steps** instead of jumping to the answer.
Like a teacher asking students to "show your work" — this helps the model:

- Break down complex problems
- Reduce calculation errors

- Improve answer accuracy

Example:

Standard Prompting vs CoT **Problem:** Roger has 5 tennis balls. He buys 2 cans of 3 balls each. How many balls now?
Standard Prompting:

```
Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each.
   How many balls now?
A: 11
```

(Model might get it wrong)
Chain of Thought Prompting:

```
Q: Roger has 5 tennis balls. He buys 2 cans of 3 balls each.
   How many balls now?
A: Let's think step by step.
   Roger started with 5 balls.
   He bought 2 cans, each with 3 balls.
   So he bought 2 × 3 = 6 balls.
   Total = 5 + 6 = 11 balls.
   The answer is 11.
```

(Showing steps dramatically improves accuracy)

Zero-shot CoT

Simplest CoT: Just add **"Let's think step by step."** at the end of your prompt.
This simple trick significantly improves performance on complex reasoning tasks!

18.2 PAL: Program-Aided Language Models

PAL Core Idea

LLMs are good at understanding problems and writing code, but bad at precise calculation.
PAL's solution:

- LLM handles: Understanding problem → Generating Python code
- Python handles: Executing code → Returning precise answer

Let each tool do what it's best at!

Example:

PAL Example **Problem:** A store sells apples for $0.75 each. If you buy 12 apples and pay with a $20 bill, how much change do you get?
LLM generates Python code:

```
apple_price = 0.75
num_apples = 12
payment = 20

total_cost = apple_price * num_apples
```

```
change = payment - total_cost
print(change)  # Output: 11.0
```

Final Answer: $11.00
LLM understands and writes code, Python ensures precision!

19 Orchestration: LLM as Reasoning Engine

Orchestration Architecture

In modern LLM applications, the LLM doesn't work alone — it serves as a **Reasoning Engine**, coordinated by an Orchestration layer:

`User Input` → **Prompt** → **LLM** → **Orchestration** → `Action/API Call`

- **LLM**: Understands intent, reasons, generates instructions/code
- **Orchestration**: Parses LLM output, executes actual actions (API calls, code execution, etc.)

Example:

Orchestration Flow **User Request:** "What's the weather in Tokyo and convert it to Fahrenheit?"
Step 1 - LLM Reasoning:

```
I need to:
1. Call weather API to get Tokyo weather
2. If returned in Celsius, use Python to convert to Fahrenheit
```

Step 2 - Orchestration Executes:

- Call Weather API: `get_weather("Tokyo")` → Returns 15°C
- Execute Python: `15 * 9/5 + 32` → Returns 59°F

Step 3 - LLM Summarizes:
"The current temperature in Tokyo is 15°C (59°F)."

19.1 ReAct: Reasoning + Acting

ReAct Framework

ReAct combines Reasoning and Acting, making the LLM alternate between:

- **Thought**: Think about what to do next
- **Action**: Execute an action (call a tool)
- **Observation**: Observe the action's result

Loop until task is complete.

Example:

ReAct Example **Question:** What is the population of the capital of France?
ReAct Process:

```
Thought 1: I need to find the capital of France first.
Action 1: Search["capital of France"]
Observation 1: The capital of France is Paris.

Thought 2: Now I need to find the population of Paris.
Action 2: Search["population of Paris"]
Observation 2: The population of Paris is approximately 2.1 million.

Thought 3: I have the answer now.
Action 3: Finish["The population of Paris, the capital of France,
          is approximately 2.1 million."]
```

19.2 LangChain: Packaging It All Together

What is LangChain?

LangChain is an open-source framework that packages all these techniques into easy-to-use modules:

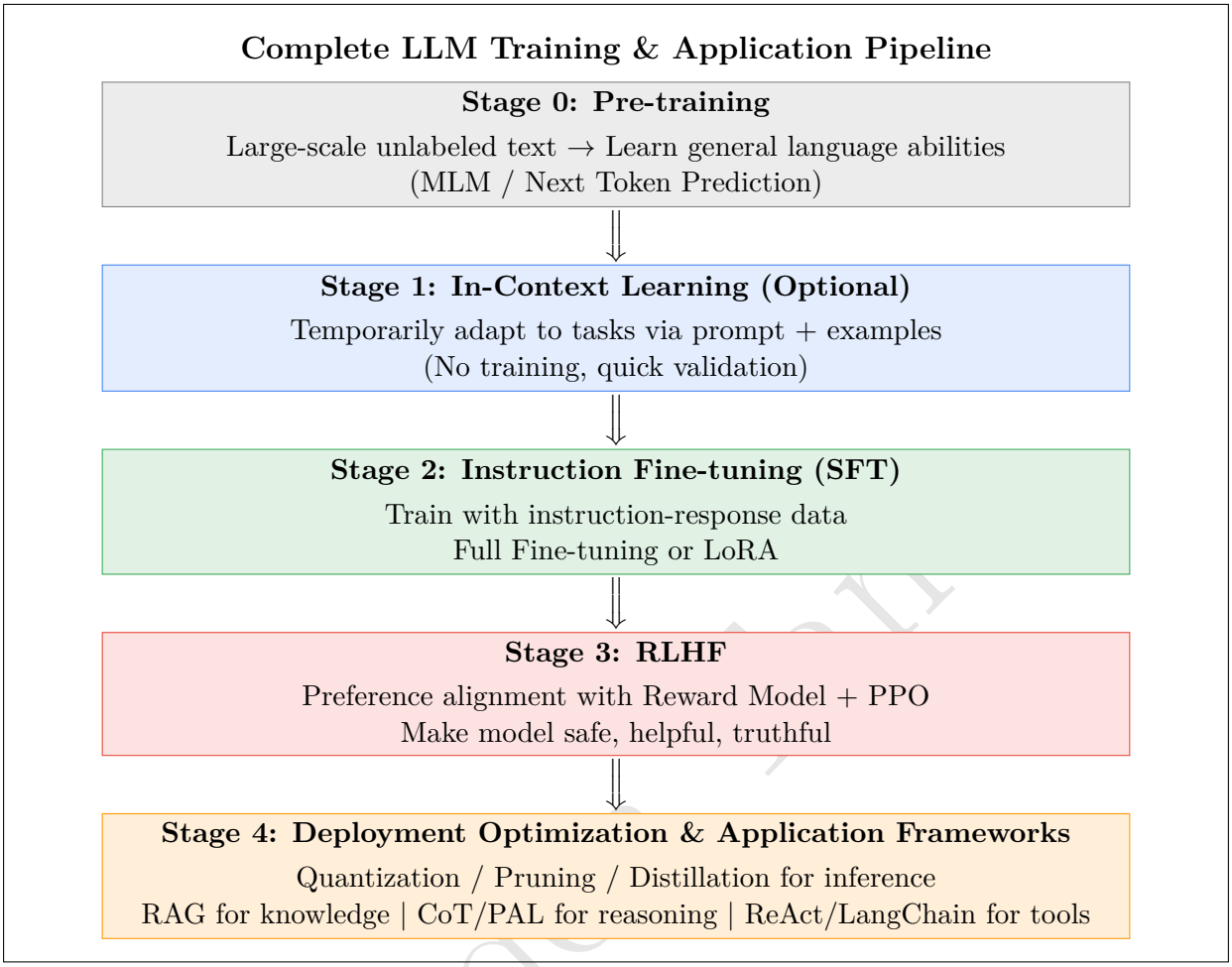
- **Models**: Connect various LLMs (OpenAI, Anthropic, local models, etc.)
- **Prompts**: Template management, few-shot examples
- **Chains**: Combine multiple steps into workflows
- **Agents**: ReAct-style autonomous decision making
- **Memory**: Conversation history management
- **Retrievers**: RAG retrieval components

LangChain Component	Corresponding Concept
`LLMChain`	Basic Prompt → LLM → Output
`RetrievalQA`	RAG implementation
`Agent + Tools`	ReAct + Orchestration
`PythonREPL Tool`	PAL (let LLM execute Python)
`ConversationBufferMemory`	Conversation history management

Week 4 Key Takeaways

1. **Optimization trio**: Quantization (reduce precision), Pruning (remove params), Distillation (train smaller model)
2. **RAG** lets LLMs access latest/private knowledge without retraining
3. **Chain of Thought** makes models show reasoning steps, improving accuracy
4. **PAL** lets LLM write code, Python executes for precise computation
5. **Orchestration** uses LLM as reasoning engine, coordinates tool execution
6. **ReAct** = Thought + Action + Observation loop
7. **LangChain** packages these techniques into an easy-to-use framework

20 Course Summary: Complete Pipeline from Pre-training to Alignment



One-Sentence Summary for Each Concept

Week 1 - Fundamentals:

- **Transformer**: Backbone of LLMs, Self-Attention is its core
- **Encoder/Decoder**: Understanding vs Generation, different architectures for different tasks
- **Pre-training**: Learn general language abilities from massive text
- **ICL**: No training, teach model via prompt examples

Week 2 - Fine-tuning:

- **SFT**: Train with ground truth, teach model to follow instructions
- **LoRA**: Train only low-rank adapters, efficient and cheap
- **BLEU/ROUGE**: Evaluation metrics based on n-gram overlap

Week 3 - Alignment:

- **RLHF**: Optimize using human preferences instead of ground truth
- **Reward Model**: The "judge" that scores responses
- **PPO**: Stable policy optimization algorithm
- **KL Penalty**: Prevents model from "cheating" or drifting too far

Week 4 - Optimization & Applications:

- **Quantization**: Reduce precision, shrink model size
- **Pruning**: Remove unimportant parameters
- **Distillation**: Small model learns from large model
- **RAG**: Retrieval-augmented generation, access new knowledge
- **CoT**: Show reasoning steps
- **PAL**: LLM writes code, Python executes
- **ReAct**: Thought-Action-Observation loop
- **LangChain**: Framework packaging all these techniques

After Completing This Course, You Should Be Able To...

1. Explain how Transformer and Self-Attention work
2. Distinguish Encoder-Only, Decoder-Only, and Encoder-Decoder models
3. Understand the roles and differences of Pre-training, ICL, SFT, and RLHF
4. Implement LoRA fine-tuning and understand its advantages
5. Explain how RLHF makes models safer and more helpful
6. Use BLEU, ROUGE, and other metrics for evaluation
7. Use Quantization, Pruning, and Distillation to optimize models
8. Build RAG systems to give models external knowledge

9. Use CoT and PAL to improve model reasoning
10. Understand ReAct framework and LangChain applications

Aiden Tan

Connect with me on LinkedIn